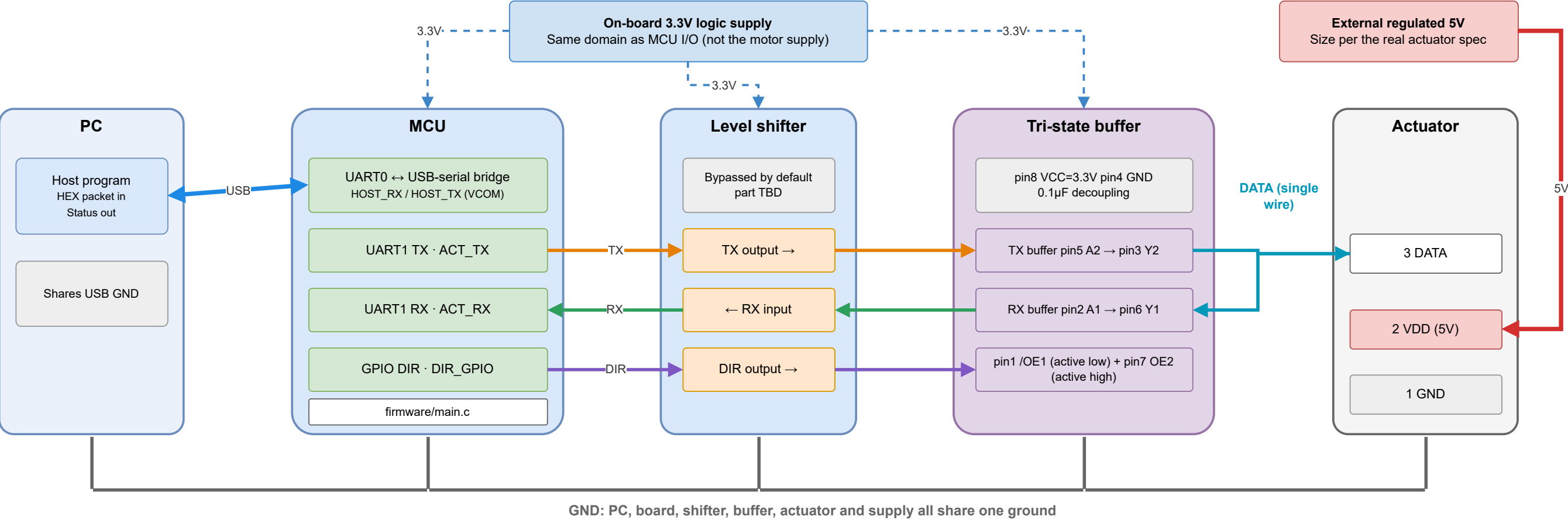


1. Hardware: PC → MCU → level shifter → tri-state buffer → actuator

Matches hardware.html. Signal names are generic teaching names, not pins of a specific board.



Legend

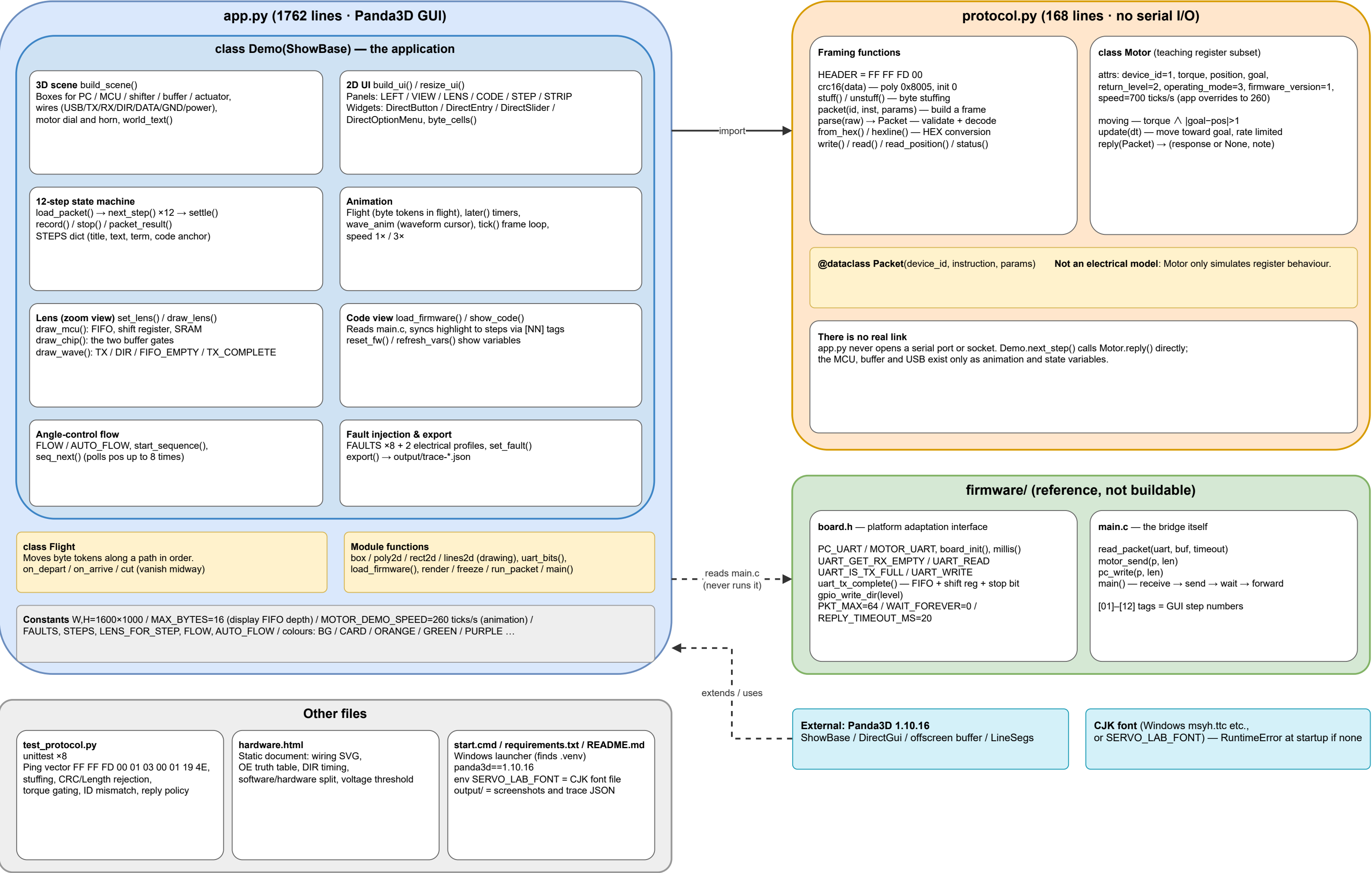
- | | |
|--------------------------|---------------------------------|
| — USB (bidirectional) | — DATA: half-duplex single wire |
| — TX: MCU → actuator | — 3.3V logic supply |
| — RX: actuator → MCU | — 5V actuator supply |
| — DIR: direction control | — GND |

Teaching assumptions (not a verified schematic)

- MCU I/O and buffer at 3.3V, level shifter bypassed by default, actuator on external 5V.
- DIR has a ~10kΩ pull-down, so the default state is "receive". Any pull-up on DATA / RX goes to 3.3V.
- Never connect the 5V supply to MCU 3.3V I/O.
- A 5V buffer needs $V_{IH} = 0.7 \times V_{CC} = 3.5V$, above a 3.3V reply, so the model defaults to a 3.3V buffer.
- Check voltages, pins and supply capacity against the datasheets of the parts you choose.

2. Software architecture: files, classes and dependencies

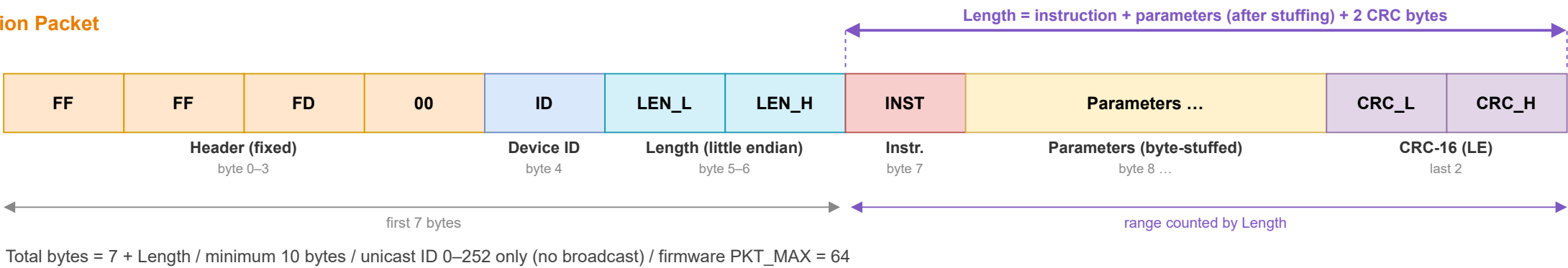
A PC-side simulator (app.py + protocol.py) and reference firmware (firmware/) meant to be ported to real hardware.



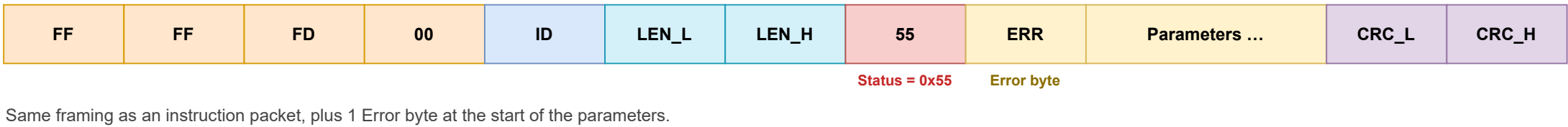
3. Packet format: layout, byte stuffing, CRC and validation order

Mirrors packet() / parse() in protocol.py. Numeric examples are computed from protocol.py.

Instruction Packet



Status Packet



Instructions supported by this simulator

INST	Name	Parameters	Response
01	Ping	none (any params → Error 5)	model number 2B =1, firmware version 1B =1
02	Read	addr (2B LE) + length (2B LE)	Error + register value
03	Write	addr (2B LE) + data (1–4B LE)	Error only (success = accepted, not arrived)
55	Status	Error (1B) + response data	actuator → PC direction only
other	Others	—	Error 2 (not implemented)

Error

Err	Meaning (used by Motor.reply)
00	Accepted
02	Instruction not implemented
04	Data out of range (Torque>1 / Return Level>2 / Mode≠3)
05	Wrong parameter / data length
06	Goal position outside 0–4095
07	Unimplemented register / mode change while torque on

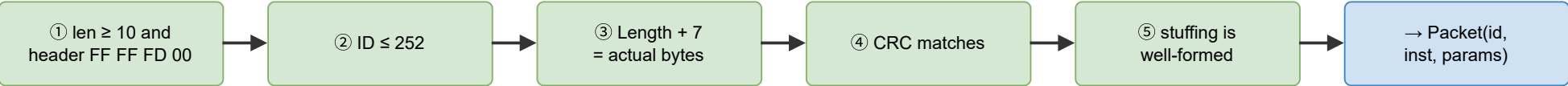
Examples (generated by protocol.py)

Case	HEX	Bytes
Ping (ID=1)	FF FF FD 00 01 03 00 01 19 4E	10
Status reply to Ping	FF FF FD 00 01 07 00 55 00 01 00 01 DB 45	14
Read addr 11 (mode)	FF FF FD 00 01 07 00 02 0B 00 01 00 22 47	14
Write addr 64 = 1 (torque on)	FF FF FD 00 01 06 00 03 40 00 01 DB 66	13
Write addr 116 = 1024 (90°)	FF FF FD 00 01 09 00 03 74 00 00 04 00 00 B2 89	16
Read addr 132 (present pos, 4B)	FF FF FD 00 01 07 00 02 84 00 04 00 1D 15	14

Byte stuffing
If FF FF FD appears in the body, insert one extra FD after it so it cannot be mistaken for a header (receiver turns FF FF FD FD back into FF FF FD).
Example: params FF FF FD → body FF FF FD **FD**, Length grows by 1.

CRC-16 polynomial 0x8005, init 0, MSB-first (non-reflected).
Computed over header ... parameters; appended **low byte first**.
Example: Ping → CRC = 19 4E

Validation order in parse() (any failure → ValueError → GUI shows "input not sent")



load_packet() additionally requires "instruction is 01/02/03" and "total length ≤ 16 bytes (display FIFO depth)".

4. Teaching motor: register map, reply policy and motion model

The Motor class in protocol.py. All of this is defined by the simulation, not a real device spec.

Register map

Addr	Name	Size	Access	Values	Behaviour	Initial
11	Operating Mode	1	R/W	3 = position control only	Change while torque ON → Error 7; value ≠ 3 → Error 4	3
64	Torque Enable	1	R/W	0 / 1	Position only moves when 1; ≥2 → Error 4	0 (OFF)
68	Status Return Level	1	R/W	0 / 1 / 2	Which instructions get a reply (table below); ≥3 → Error 4	2
116	Goal Position	4	R/W	0 – 4095	≥ 4096 → Error 6. With torque OFF it is stored but nothing moves	2048 (180°)
122	Moving	1	R	0 / 1	torque ∧ goal – position > 1 (readable only)	0
132	Present Position	4	R	ticks	Current position, read with Read(132,4)	2048 (180°)

Reply policy (Status Return Level)

Level	Ping	Read	Write
0	✓	✗	✗
1	✓	✓	✗
2 (default)	✓	✓	✓

Whether a Write replies depends on the Level **before** the write (a Write that changes 2→1 is still answered). Error replies follow the same policy (Ping length error is always answered).

Angle ↔ ticks
ticks = angle × 4096 ÷ 360 (GUI: round() mod 4096)
angle = ticks × 360 ÷ 4096
e.g. 90° = 1024, 180° = 2048, 359° = 4085
Goal is 4 bytes, little endian (90° → 00 04 00 00)

Motion model Motor.update(dt)
when torque = 1
position += clamp(goal – position, ±speed × dt)
speed = 700 ticks/s (Demo sets 260 ticks/s to slow the animation)
when torque = 0 the position never moves

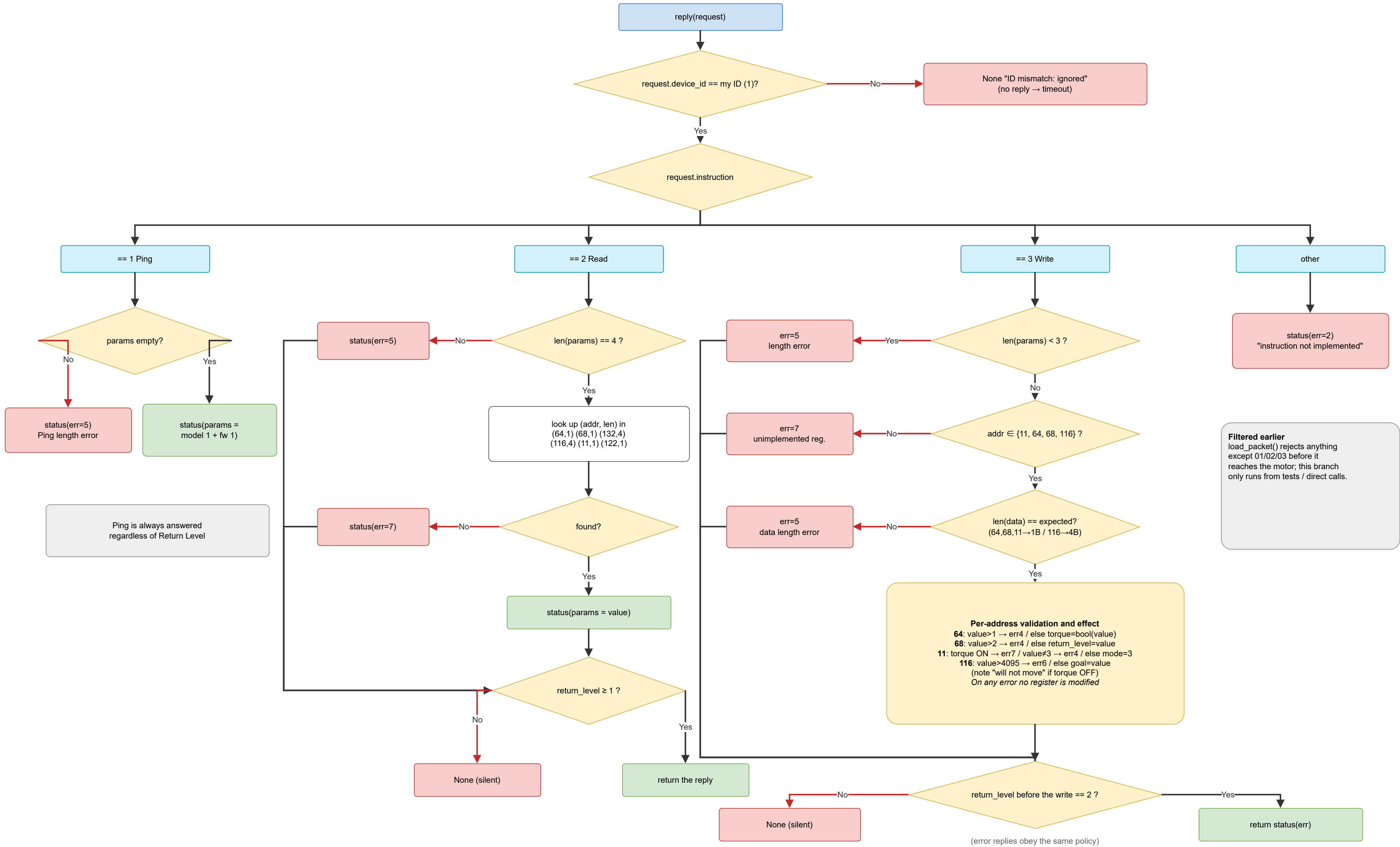
Note: Write success ≠ arrival Error = 0 only means the instruction was accepted. Arrival is checked by reading Present Position (132) and comparing with the goal (|Δ| ≤ 2 ticks). In the GUI the position advances every frame in tick(), so a Read right after a Write returns a mid-move value.

Ping reply params = 2-byte model number (LE, value 1) + 1-byte firmware version (value 1). Both are **simulated values**, not detected from a real device. Ping always replies regardless of Return Level.

(address, length) pairs supported by Read (64,1) (68,1) (132,4) (116,4) (11,1) (122,1) — anything else → Error 7.
ID matching if request.device_id ≠ 1 the whole packet is ignored (no reply). The GUI fault "ID mismatch" rewrites the ID to 2 to reproduce this.

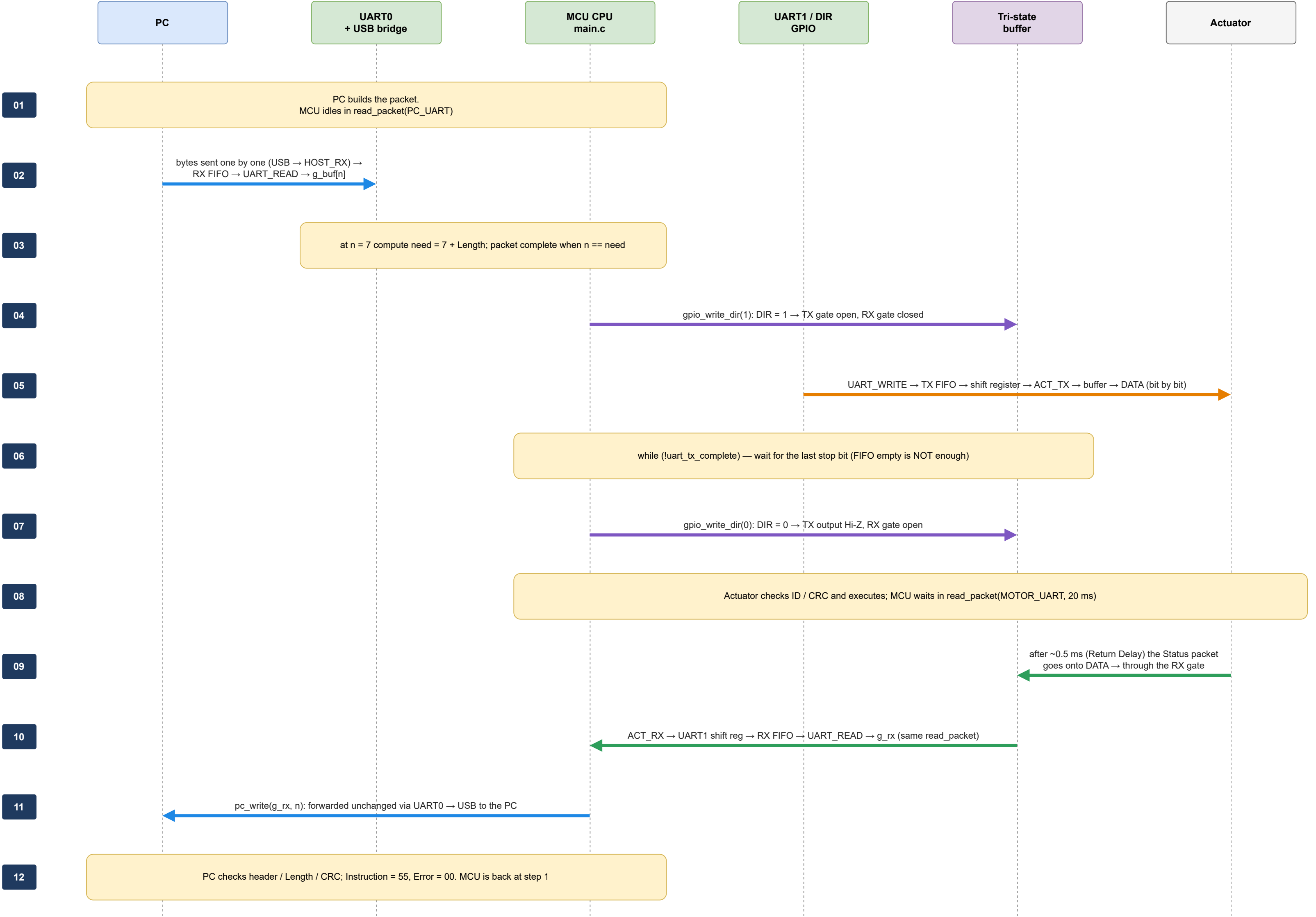
5. Simulated motor logic: Motor.reply(request)

Returns (response or None, note). None means nothing is sent (the MCU times out after 20 ms).



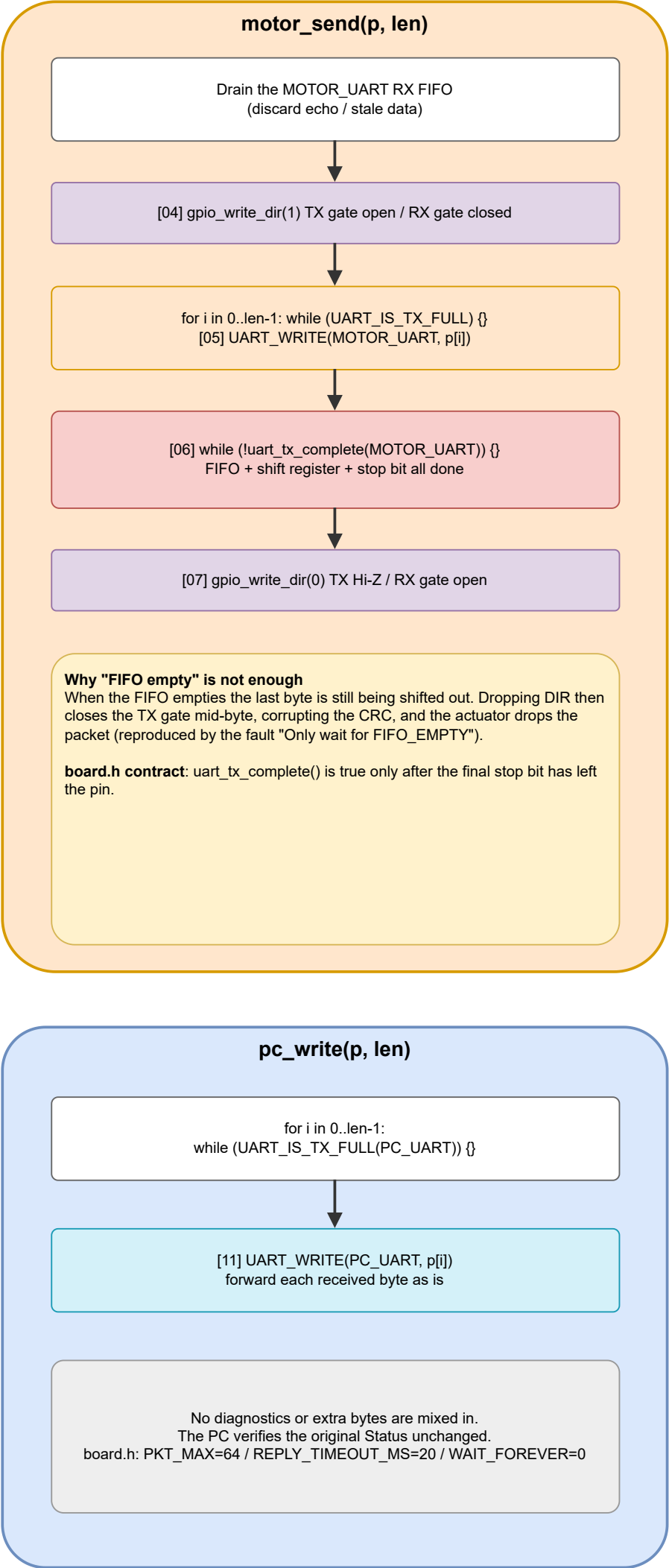
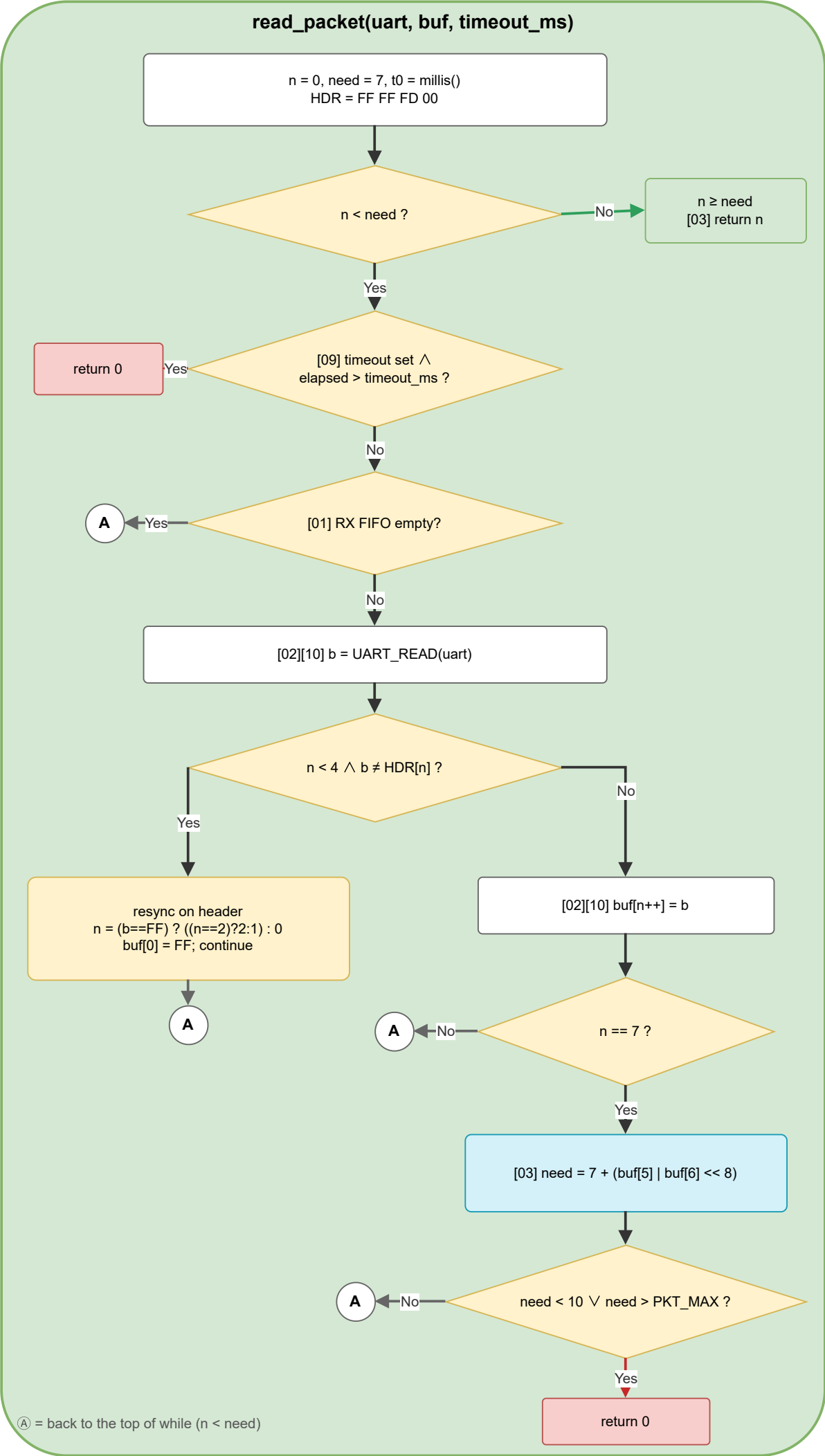
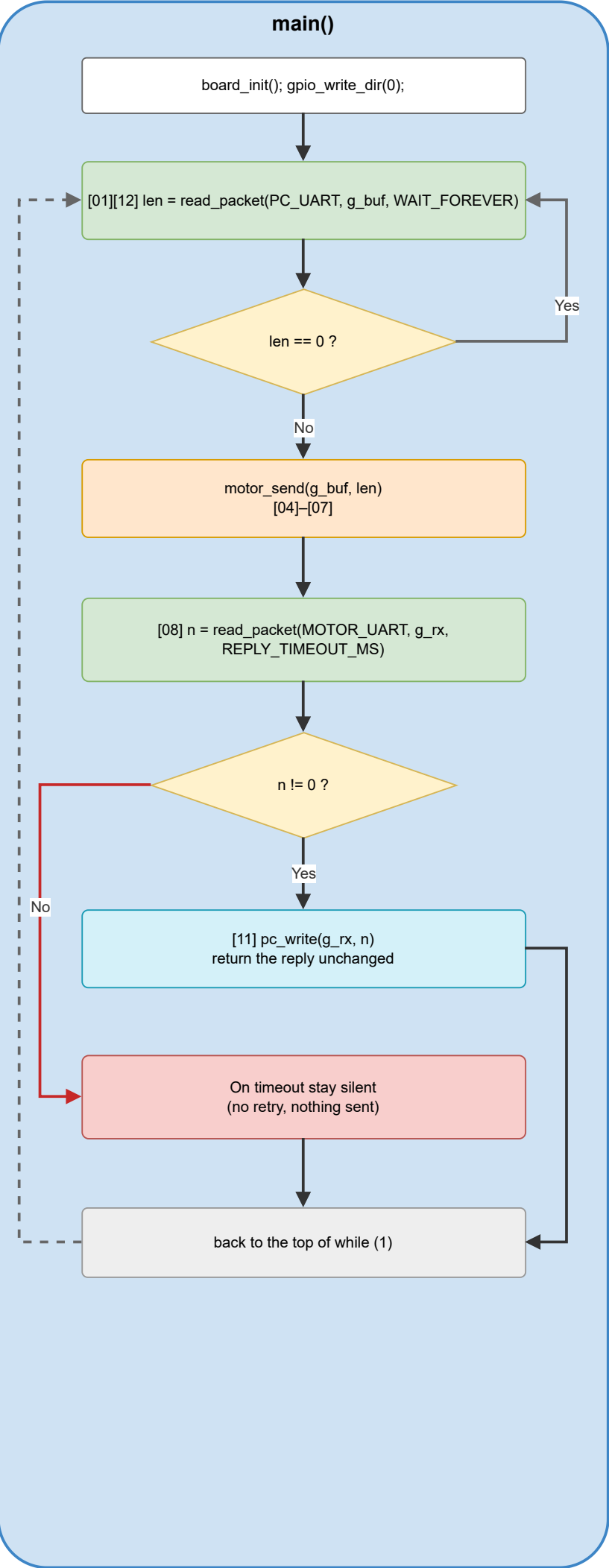
6. The 12-step sequence for one packet round-trip

One click of "Next" = one step. Each number matches the [NN] comment tags in firmware/main.c.



7. firmware/main.c: main() / read_packet() / motor_send() / pc_write()

[NN] are the step tags in the code comments. Left: main loop / middle: receive / right: send and forward. A circled "A" jumps back to the top of while (n < need).



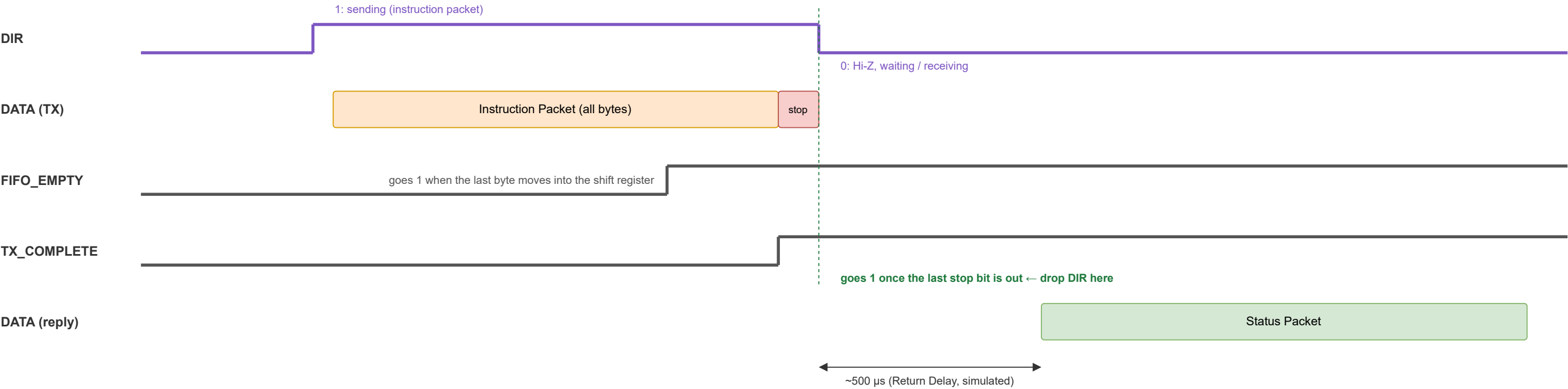
8. One DIR line drives both gates: truth table and timing

Corresponds to "one GPIO controls two buffers" in hardware.html and steps [04]–[07].

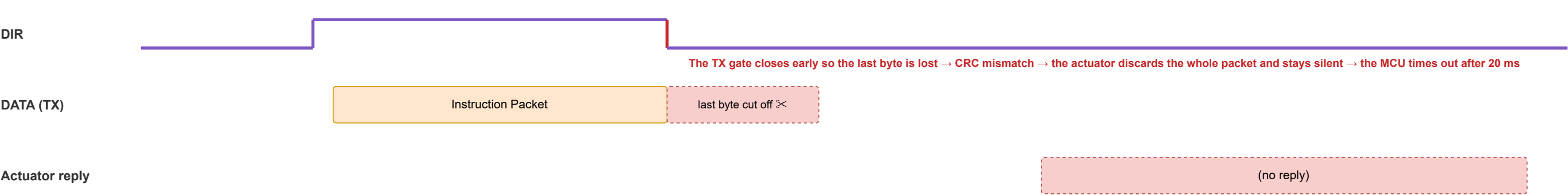
DIR	pin 1 /OE1 (active low)	pin 7 OE2 (active high)	Result
1 / High	High → RX gate closed	High → TX gate open	MCU TX → DATA → actuator (sending the instruction)
0 / Low	Low → RX gate open	Low → TX gate closed (Hi-Z)	actuator DATA → MCU RX (receiving the Status)

Hi-Z = the output is "unplugged": it drives neither 1 nor 0 and leaves DATA free. The chip knows nothing about Ping, CRC or when a frame ends. Switching DIR is the MCU firmware's job.

Correct timing (as in firmware: wait for `uart_tx_complete`)



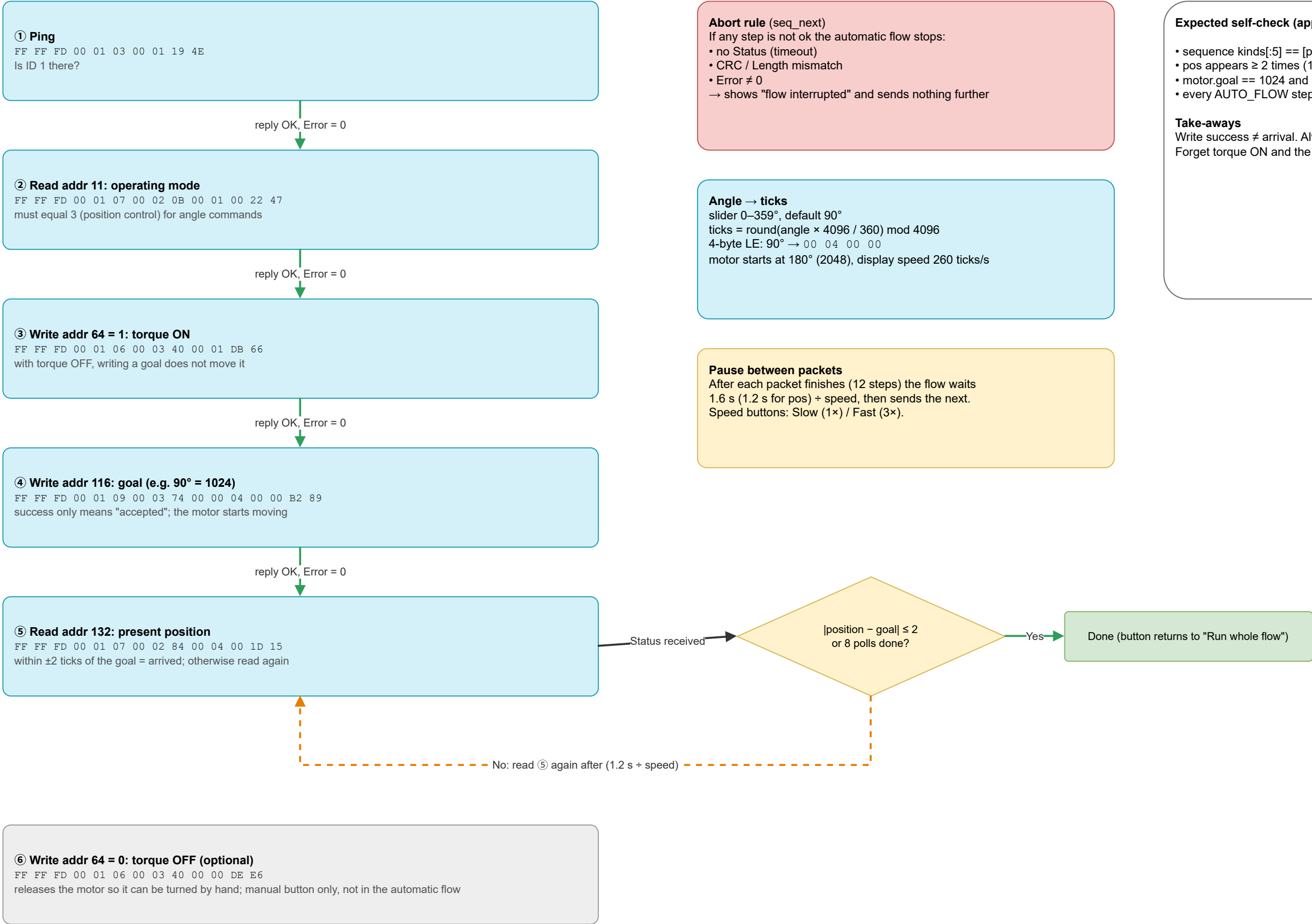
Wrong timing (dropping DIR as soon as `FIFO_EMPTY`)



Another failure: DIR never returns to 0 The TX gate keeps occupying DATA and the RX gate stays closed. The actuator replies but the MCU never hears it → 20 ms timeout.
Verify on hardware Probe DATA and DIR_GPIO with an oscilloscope and check that the DIR falling edge lands after the last stop bit. The GUI waveform is a teaching sketch, not a measurement.

9. Turning to a target angle: the automatic flow (Ping → mode → torque on → write goal → poll position)

FLOW / AUTO_FLOW in the left panel. Each button sends a single packet; the automatic flow runs ①–⑤.



10. Fault injection (FAULTS ×8) and electrical profiles (×2)

Chosen in the left-panel drop-downs. The choice is latched when a packet is loaded (active_fault).

Fault (GUI label)	How it is simulated	Step affected	Outcome
Normal	Nominal	1 → 12 complete	Status received, CRC OK
Only wait for FIFO_EMPTY, then switch DIR	[06] replaced by a FIFO_EMPTY wait	6: red line on waveform / 7: DIR drops early / 8: no reply	last byte truncated → CRC mismatch → actuator discards → 20 ms timeout
DIR not returned to receive	[07] skipped (DIR stays 1)	7: line skipped / 9: reply blocked at the gate	actuator replies but the MCU never hears it → timeout
Actuator unpowered	no external 5V	8: no reply	MCU waits 20 ms, read_packet returns 0, nothing reaches the PC
No common ground	no shared voltage reference (treated as failure)	8: no reply	real behaviour is undefined; shown as failure
ID mismatch	load_packet() rewrites the ID to 2	8: actuator ignores it	Motor.reply() returns None → no reply
CRC corrupted in transit	a byte is garbled on the wire	8: no reply	CRC mismatch → neither executes nor replies
Write returns no Status	return_level forced to 1 (policy_override)	8: write executes, nothing after	"executed, no reply by setting"; restored to 2 on the next load_packet()
Electrical: example 5V buffer (to verify)	VIH = 0.7×5V = 3.5V > a 3.3V reply	8: stops with a risk warning	"cannot judge reliability" — a conservative warning, not a measured failure

Common to all faults stop() sets finished=True, running=False, shows "timeout: no valid Status" and marks the flow step failed, aborting the automatic flow (only the "Write returns no Status" case is marked ok).

Tip Press Reset before an experiment. A fault only takes effect when the next packet is loaded.

Known behaviour "ID mismatch" makes Motor.reply() return None, so the screen takes the same path as "no reply due to Status Return Level" (shown as executed / ok).

11. Simulator window (1600×1000 shown at 0.55×)

Matches the rectangle constants LEFT / VIEW / LENS / CODE / STEP / STRIP in app.py.

Serial Actuator Communication Lab

Hardware, MCU internals and firmware code stay in sync

Simulation mode · no hardware

01 Turn to angle

slider (0–359°)
[Run whole flow]
[Slow][Fast]

① Ping
② Read 11
③ Write 64=1
④ Write 116
⑤ Read 132
⑥ Write 64=0

Fault menu
Electrical profile

Motor dial
(current / goal)

02 Hardware (3D) ←/→ to rotate

PC — USB — MCU — level shifter — buffer — actuator
USB blue · TX orange · RX green · DIR purple · DATA cyan
flying squares = bytes in transit

bottom badge: DIR HIGH/LOW and gate state

03 Firmware main.c

where: main() → read_packet(...)

lines tagged [NN] highlighted yellow
auto-scrolls with the step

Lens (zoom) [MCU internals] [Buffer] [Waveform]

MCU: UART0/UART1 shift registers, FIFO(16), SRAM g_buf/g_rx, DIR
Buffer: TX/RX gates, /OE1 and OE2
Waveform: TX, DIR, FIFO_EMPTY, TX_COMPLETE

Controls & explanation

HEX entry (editable)
[Load][Next][Auto play][Replay][Reset]

step heading 01/12 ...
plain-language text + glossary

Byte strip TX PC→actuator (20 cells) / RX actuator→PC (20 cells) decode: ID · instr · length · CRC OK / Error

Lens tab chosen per step (LENS_FOR_STEP)

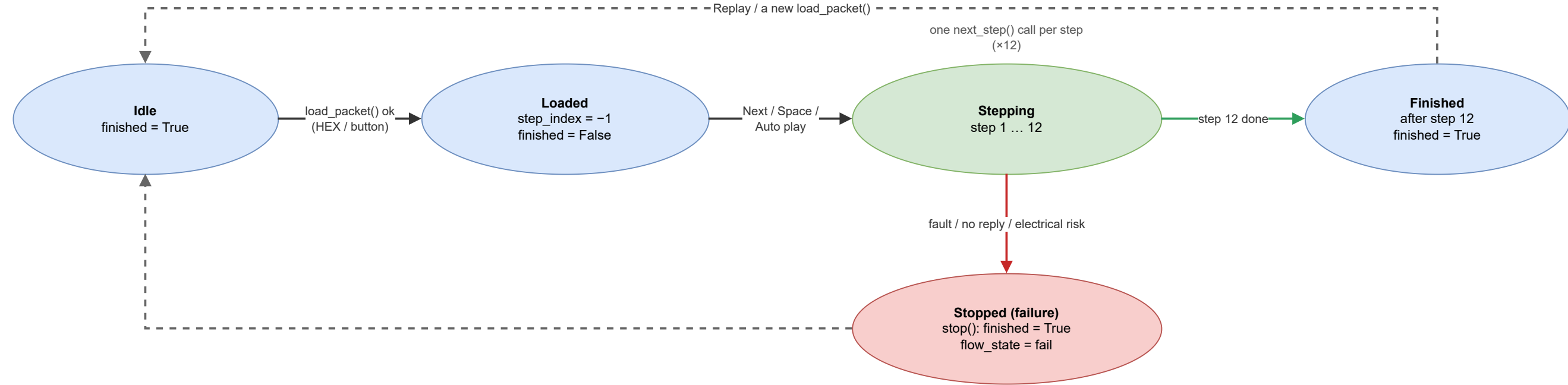
Lens tab	Steps
MCU internals	1, 2, 3, 5, 8, 10, 11, 12
Tri-state buffer	4, 7, 9
Waveform	6

Keyboard & behaviour

- **Space** = next step (next_step)
- ←/→ = rotate the 3D camera ±8°
- **Esc** = quit
- follows window resizes (resize_ui)
- **--screenshot** = headless self-test + saves output/*.png
- trace export() → output/trace-YYYYMMDD-HHMMSS.json
- mode = SIMULATION_ONLY, hardware_tested = false
- HEX input limited to 16 bytes (display FIFO depth), instructions 01/02/03 only

12. Runtime: packet state transitions and the tick() frame loop

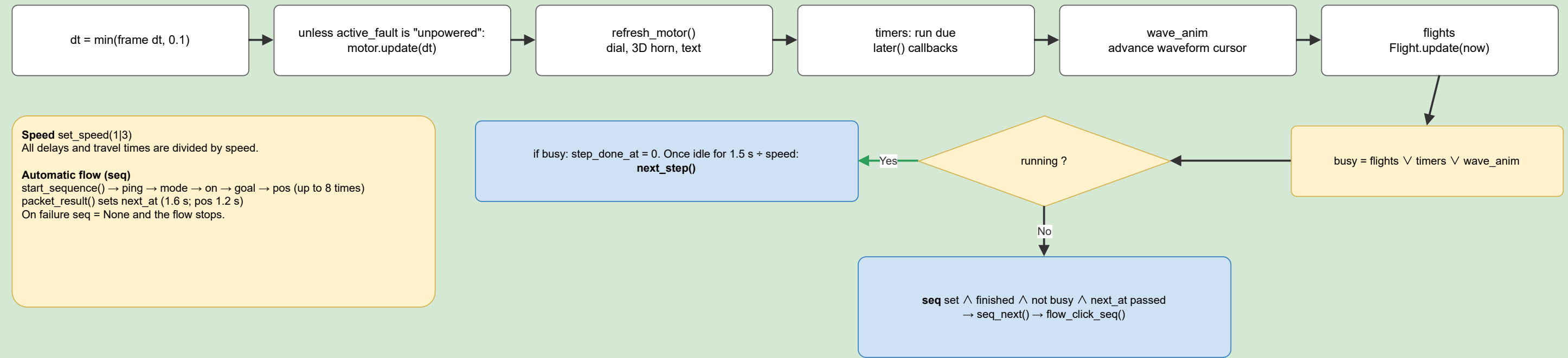
How Demo.finished / running / step_index / seq relate.



load_packet() input checks
from_hex() → parse() (CRC / Length / header) → instruction ∈ {1,2,3} → length ≤ 16
On failure shows "input not sent" and leaves state unchanged. On success discards flights, latches active_fault / active_profile, adjusts return_level, resets FW variables and shows the decode line.

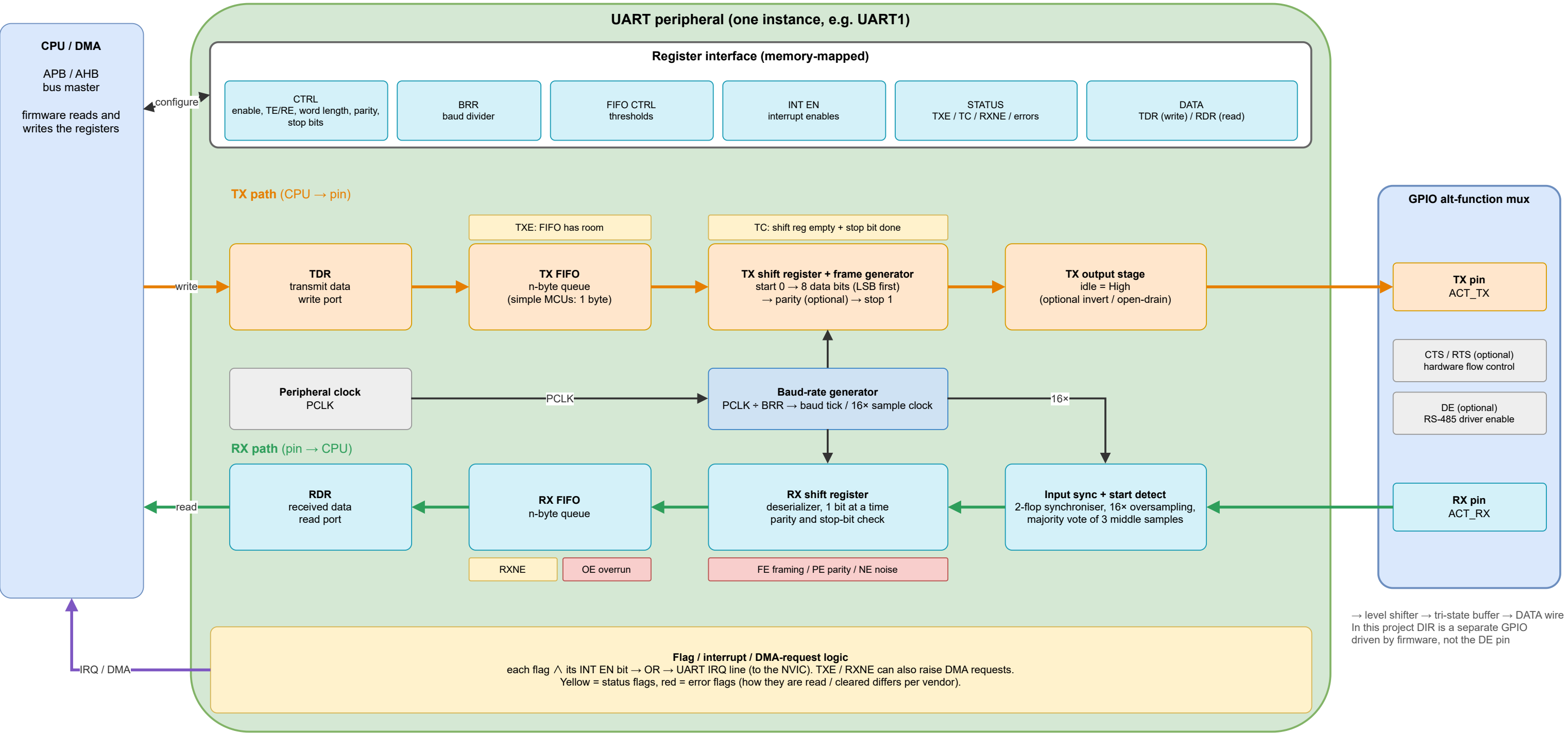
settle(): before advancing, instantly finishes flights, timers and waveform animation (up to 200 passes).

tick(task) — runs every frame (taskMgr)

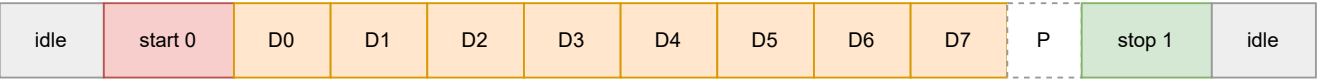


13. Internal structure of a typical MCU UART (generic model)

Register names, FIFO depth and flag names differ by vendor. This shows the blocks most MCU UARTs have in common.



One frame on the wire (8N1)



Data is sent LSB first. P is the optional parity bit (absent in 8N1). At 57,600 bps one bit ≈ 17.4 μs and one byte (10 bits) ≈ 174 μs. uart_bits() in app.py returns the same 10 bits.

"FIFO empty" ≠ "transmission complete"
When the TX FIFO is empty (TXE) the last byte is **still inside the TX shift register** being sent. TC is set only after both the FIFO and the shift register are empty and the last stop bit has left the pin. Half-duplex DIR switching must wait for TC (firmware/main.c [06]).

This project's board.h API	What it maps to in the UART
UART_GET_RX_EMPTY(uart)	RX FIFO empty (RXNE = 0)
UART_READ(uart)	read RDR = pop one byte from the RX FIFO
UART_IS_TX_FULL(uart)	TX FIFO full (TXE = 0)
UART_WRITE(uart, b)	write TDR = push one byte into the TX FIFO
uart_tx_complete(uart)	TC flag (FIFO + shift reg empty + stop bit done)

The "MCU internals" lens in the GUI (draw_mcu) shows the FIFO (16 slots), shift register and SRAM as a simplified version of this structure.